

Deep Neural Network for Garbage Image Classification

A Transfer-Learning Approach with a ResNet-18 Backbone

Gianmarco Costanzo (34873A)

June 26 2026

Abstract

This paper is made for the «Algorithms for Massive Data» exam module. The task was to implement a Deep Neural Network (DNN) classifying images contained in the Garbage dataset, published under the MIT license. I chose to use one of the simplest pre-trained models that have a closely matching training resolution: the ResNet models pre-trained on ImageNet expect input images resized to 224×224 pixels. The general structure of this lightweight CNN is made of four central steps (layer1–layer4), preceded by an input block and followed by a final head classifier. I focused on the “head” and on the layer4 in two separate training code blocks, to gain model flexibility without increase the training time too much. The final result, run on a completely unused test image set, reached a test accuracy of 92.3% and a test loss of 0.2405.

I. Introduction

Automatic waste classification is studied to help and optimize recycling and garbage-sorting facilities. In machine-learning terms, this could be translated into a multi-class image classification task: given an input image, the model must assign it to a single waste category, Ten in this particular dataset. I chose ResNet-18 for two different reasons: first, as I anticipated, to keep an acceptable training time so that I could focus more on the hyperparameter selection, in order to show exactly how much the hyperparameters influence the result while keeping the model fixed. second, to avoid starting from scratch by reusing pre-trained features. Transfer learning is fundamental, especially when the training set we work on is not large enough to build an entire complex CNN from zero. To solve this task and to improve the initially low values of validation accuracy, I decided to train the network in two stages, as first I build a grid search over four hyperparameter configurations to understand which combination could have the better response without using a more complex Cross-Validation. In the second stage I focus only on the learning rate, selected on the validation set, after enabling the fine-tuning of `layer4`. This procedure allowed me to go from an initial $\sim 87\%$, obtained by tuning only the classifier head, to more than 92% of test accuracy. The procedure presented in this paper is fully reproducible: the «Garbage dataset» is published under the MIT license, and the entire Python `.ipynb` notebook will be attached, according to the exam rules, in my GitHub repository. The connection to Kaggle is built without any manual credential setup through the `kagglehub` library, for both simplicity and anonymity.

II. Dataset and Data Organisation

1 Dataset Description

The dataset is available in three directories with different image resolutions. The directories contain the same images and the same number of observations, making it possible to select the resolution that best matches the input requirements of the chosen model. I used the 256-pixel version because

it is the closest to the 224×224 input expected by ResNet-18. Every image directory is composed of the same 10 classes, namely: battery, biological, cardboard, clothes, glass, metal, paper, plastic, shoes, and trash. The total amount of images available at the moment I completed my DNN is 12,259. From the Kaggle notebooks and publications I reviewed, the total number of images appears to have changed over time. This should be considered when comparing reported accuracies, together with differences in model architecture, training procedure, and available computational resources. A descriptive analysis of the dataset composition immediately highlights how the classes are relatively unbalanced, with trash at the bottom of the per-class count at just $\sim 4\%$ of the entire dataset. This is reasonable, in line with the general tendency to reduce unrecyclable garbage.

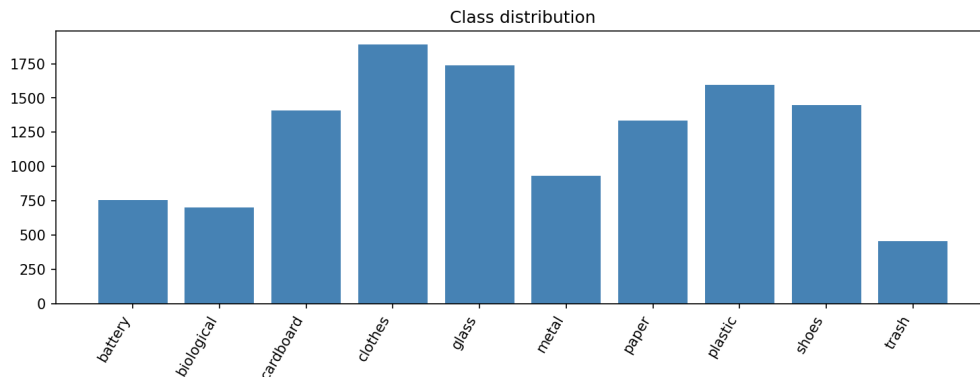


Figure 1: Number of images per class. The dataset is moderately unbalanced, with trash the least represented class ($\sim 4\%$) and clothes the most frequent.

2 Train/Validation/Test Split

In this subsection I explain how the training, validation and test sets are built from the original dataset. First of all, I decided to use `trainTestSplit` on the image paths and labels, so that no single image can be randomly assigned to more than one set. I found this error to be common in one of the most upvoted Kaggle works. Splitting on the paths, rather than relying on `randomSplit` over a single dataset object, gives me control of the distribution and over what ends up in each set. On the same time I assign different transformations to each split: data augmentation on the training set, and only resizing and normalization on validation and test, which must stay deterministic. With `RandomSplit` over one shared dataset object this separation is not possible, because all subsets inherit the same transformations. Going deep down I used the training to update the weights and the validation one for model selection and early stopping. The test set is touched only once, at

the very end, to obtain an unbiased estimate of the model’s generalization. Keeping a fresh test set is essential: reusing the validation set for testing is one of the most classic methodological errors, and it leads unreliable estimate of performance. I chose an 80-10-10 split so that the training set is large enough to learn from, while keeping the validation and test sets of equal size, large enough to produce stable and comparable performance estimates. The last important parameter in the split configuration is `RandomState`, set to 42, which makes the split reproducible by fixing the randomness of the partitioning so that the same division is obtained at every run.

3 Preprocessing and Data Augmentation

Normalization and resizing are shared between the `TrainTransform` and the `ValTransform` functions, while augmentation instead is applied only to the training set. This approach guarantees with that the validation and test sets stayed deterministic, and that the same image stays comparable across runs. The augmentation includes classical transformations such as `RandomHorizontalFlip` and `RandomRotation`, training set is lastly enhanced with different `Brightness`, `Contrast` and `Saturation`.

III. Method

1 Model Architecture

Now I go with a deeper look at the use of ResNet-18. The first action I took was to fix a batch size and complete the implementation of the three sets via `DataLoader`, with shuffling disabled on the validation and test sets to avoid random sampling during batch composition. I then built the real model code block, initially with the backbone frozen so I had to train only the new classifier head that replaces the original fully connected layer. The head starts with a Dropout layer, the operation that turns off neurons with probability fixed based on the fixed values of the hyperparameters grid at every forward pass. It is followed by a linear layer that maps the 512 input features produced by the backbone to a 256 dim each of them weighted linear combination of all 512 inputs ($y=Wx+b$), with weights initialized randomly and then learned during training. The result passes through a ReLU activation, which introduces the non-linearity that allows the hidden layer to learn non-trivial representations; without it, the two linear layers would collapse into a single linear transformation. A second Dropout follows, and a final linear layer maps the 256 features to the 10 output classes. With the backbone frozen, only the new classifier head is trainable: 133,898 parameters out of the

~11.7M of ResNet-18, roughly 1.1% of the network.

2 Training Procedure

The training is organized as a for cycle over epochs, where a single epoch is a complete pass over all the images, processed in batches of 32 at a time. After some testing I chose to set the number of epochs to 25. Related to it is the hyperparameter *patience*, which defines the maximum number of consecutive epochs without an improvement in the validation loss: when this limit is reached, an early stop is triggered. As a loss function I used `CrossEntropyLoss`, this criterion already applies the softmax internally, so the network head outputs raw logits and no explicit softmax layer is added. The optimizer is Adam, applied only to the parameters with `requires_grad` set to `True`: in this way the update touches only the trainable part of the network. Last Hyperparameter is a weight decay term is used as L2 regularization. The actual learning happens through gradient descent, repeated once per batch: `loss.backward()` computes, for every trainable weight, the gradient of the loss with respect to that weight, and `optimizer.step()` then updates the weights by moving them a small step in the direction suggested by those gradients. The size of this step is controlled by the learning rate, which is exactly the hyperparameter I later focus on during fine-tuning. To improve convergence I try to introduce a scheduler that halves the learning rate when the validation loss stops improving. The training keeps track of the epoch with the lowest validation loss and, at the end of the run, restores those weights, so that the model returned is the best one found and not the one from the last epoch.

IV. Results

1 Model Selection Results

Table 1 reports the results of the learning-rate ablation on the fine-tuning of `layer4`. The three learning rates produce clearly different behaviours. With 10^{-3} the training is unstable: the validation loss oscillates during training, with a sharp spike around the eighth epoch, and the model starts overfitting early, reaching its best validation loss of 0.3388 already at epoch 6. The step size is simply too large for the already well-tuned convolutional weights of `layer4`, which get pushed too far at each update. With 10^{-4} the curve is much cleaner and the validation loss decreases smoothly to 0.2900 at epoch 5; after that, however, the model begins to overfit, with the training accuracy climbing while the validation loss rises again. This learning rate converges quickly but leaves little

room before overfitting sets in. With 10^{-5} the convergence is slower, as expected from smaller update steps, but the model keeps improving for many more epochs without overfitting, with the training and validation curves staying close to each other. It eventually reaches the lowest validation loss of the whole ablation, 0.2678 at epoch 18, with a validation accuracy of 0.9225. The slower, smaller steps act as an implicit form of regularization, letting the model settle into a better minimum before it starts to memorize the training set. For these reasons I selected the 10^{-5} model as the final candidate.

Table 1: Fine-tuning di layer4: ablation sul learning rate (dropout = 0.3, weight decay = 10^{-4}).

Learning rate	Best Val Loss	Val Accuracy	Best epoch
1×10^{-3}	0.3388	0.9054	6
1×10^{-4}	0.2900	0.9260	5
1×10^{-5}	0.2678	0.9225	18

2 Final Test-Set Performance

Once the best model was selected on the validation set, I evaluated it a single time on the held-out test set, which had never been used during training or model selection. A particularly reassuring observation is that the test loss (0.2405) is actually lower than the validation loss of the selected model (0.2678), and the test accuracy (92.33%) is essentially the same as the validation accuracy (92.25%) this is the opposite of the usual drop one observes when moving from validation to test, and it indicates that the model did not overfit the validation set during model selection: the performance estimated on data used for the selection is confirmed on completely unseen data. In other words, the generalization capability of the network is solid, and the result obtained on the test set can be trusted as an unbiased estimate of how the model behaves on new images.

3 Per-Class Analysis

Beyond the aggregate accuracy, Table 2 reports precision, recall and F1-score for each of the ten classes, which is essential given the class imbalance discussed earlier. The performance is far from uniform across categories. The strongest classes are clothes and shoes, both with an F1-score around 0.99, and battery, which reaches a perfect recall of 1.000 these are visually distinctive objects, easy to separate from the rest. The weakest class is by far trash, with a recall of only

0.778 and an F1-score of 0.833. This is not surprising: `trash` is both the smallest class, with just 45 images in the test set, and semantically the vaguest one, since it collects mixed waste that does not belong to any specific material category, and is therefore easy to confuse with everything else. The imbalance and the ill-defined nature of the class reinforce each other. Other comparatively weak classes are `metal` (F1 0.845) and `plastic` (recall 0.849). Looking at the errors, the most plausible confusions are between materials that look alike: `plastic` and `glass`, which share transparency and reflective surfaces, and `paper` and `cardboard`, which are essentially the same material at different thicknesses. These confusions are intuitive and reflect genuine visual similarity rather than a failure of the model.

Table 2: Per-class performance on the test set.

Class	Precision	Recall	F1-score	Support
battery	0.927	1.000	0.962	76
biological	0.957	0.957	0.957	70
cardboard	0.935	0.915	0.925	141
clothes	0.989	0.995	0.992	189
glass	0.878	0.914	0.896	174
metal	0.840	0.849	0.845	93
paper	0.877	0.903	0.890	134
plastic	0.900	0.849	0.874	159
shoes	0.993	0.986	0.990	145
trash	0.897	0.778	0.833	45
Accuracy				0.923
Macro avg	0.919	0.915	0.916	1226
Weighted avg	0.924	0.923	0.923	1226

V. Conclusion

Beyond the numerical result, the main lesson of this work is methodological: a clean model selection performed on the validation set, together with an evaluation carried out only once on an isolated test set, produced a generalization estimate that proved reliable, with the test performance even slightly better than the validation one. Also the use of lightweight CNN model does not lead to result so different from more powerful elaboration in this topics leading to complete a test with over 90% of accuracy with just 12k images.